# Q-learning in one maze

Name: Lukasz Baldyga Student ID: B24D-5026

## 1 Implementation:

The Q-learning is implemented in **Rust** language with some Python libraries for visualization.

The code is in: https://github.com/dashingcontrol/q-learning-rust

The main file with the algorithm (written as a learning practice for myself in rust) is in main.rs file.

## 2 Introduction

The agent learns a route through one $6 \times 6$ maze. An open cell is a state. The available actions are up, down, left, and right, but the program selects only moves that lead to another open cell. The start is $S = (5, 0)$ and the goal is $G = (0, 5)$. Each move receives reward $-1$, so a shorter successful route has a larger return.

Rust handles training and CSV export. Python only reads the saved data and draws the figures. Following slide 40, the program records Q-values before training, after episode 50, and after episode 100. It also saves the training paths from episodes 1, 10, and 100.

**Table 1:** Parameters and the reason for using them in this example.

| **Setting** | **Value** | **Reason** |
|---|---|---|
| Episodes | 100 | The assignment asks for the 100th training path. |
| Learning rate | $\alpha = 0.40$ | Changes the table within a short run while retaining part of the old estimate. |
| Discount | $\gamma = 0.90$ | Passes future step costs back through the maze. |
| Exploration | $\max(0.40 \cdot 0.97^{e-1}, 0.02)$ | Explores more at the start and becomes more consistent later. |
| Reward | $-1$ per legal move | Gives shorter successful routes a better return. |
| Episode limit | 200 steps | Stops an unsuccessful episode from running forever. |
| Random seed | 7 | Makes the example repeatable. |

## 3 Q-learning method

For a transition from $s$ to $s'$ after action $a$, the code applies the off-policy Q-learning update from the lecture:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a' \in A(s')} Q(s',a') - Q(s,a) \right]. \tag{1}$$

Only permitted actions are included in $A(s')$. At the goal, the future term is zero. For the first update, all stored values are zero, so

$$Q_{\text{new}} = 0 + 0.40[-1 + 0.90(0) - 0] = -0.40.$$

Training uses epsilon-greedy action selection. Epsilon begins at 0.40, is multiplied by 0.97 after each episode, and does not fall below 0.02:

$$\varepsilon_e = \max(0.40 \cdot 0.97^{e-1}, 0.02).$$

Most moves therefore use a current best action, while some moves explore. Equal best actions are chosen randomly during training. The final policy check sets $\varepsilon = 0$.

## 4 Training results

The first episode takes 36 steps. Episode 10 takes 22, and episode 100 takes 10. The first ten episodes average 51.6 steps; the last ten average 10.2. All 100 training episodes reach the goal, and the first 10-step training route appears at episode 30.

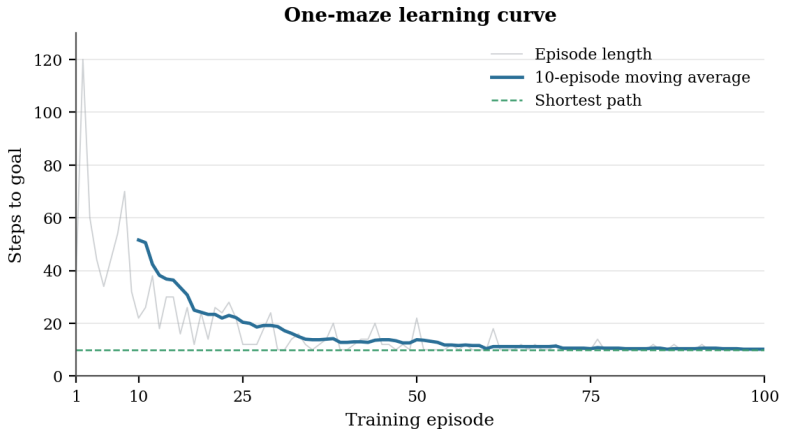


**Figure 1:** The light line shows every episode. The darker line averages the latest ten episodes.

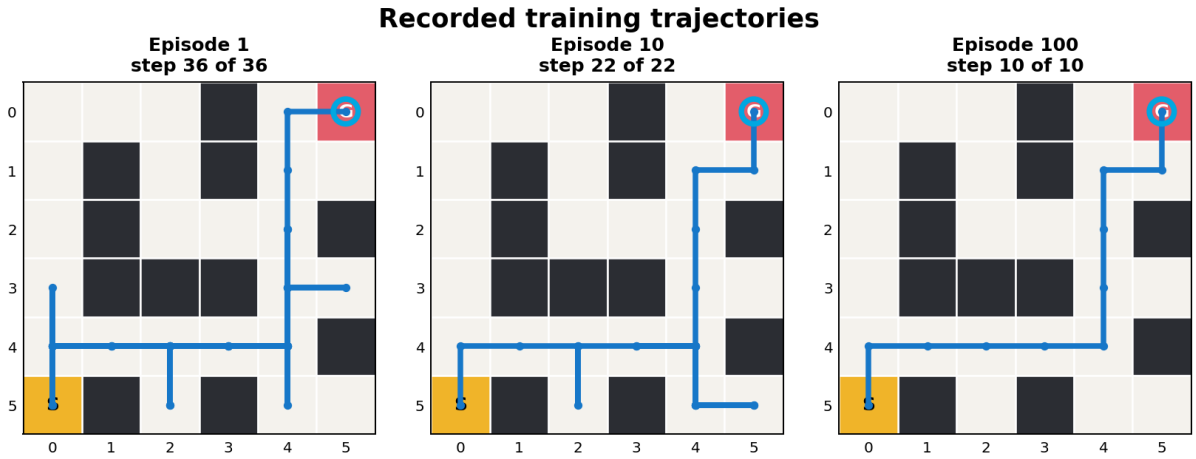


**Figure 2:** Training paths saved at episodes 1, 10, and 100. Repeated segments come from exploration.

Figure 2 shows how the route changes during learning. In episode 1 the agent revisits several cells and needs 36 steps to reach the goal. By episode 10 it follows most of the useful route, but exploratory moves create detours and the path still takes 22 steps. Episode 100 reaches the goal in 10 steps, which is the shortest possible path length for this maze.

These paths include the exploration used during training. The separate evaluation on page 3 sets epsilon to zero and follows only the best actions stored in the final Q-table.

## 5 Q-values and final policy

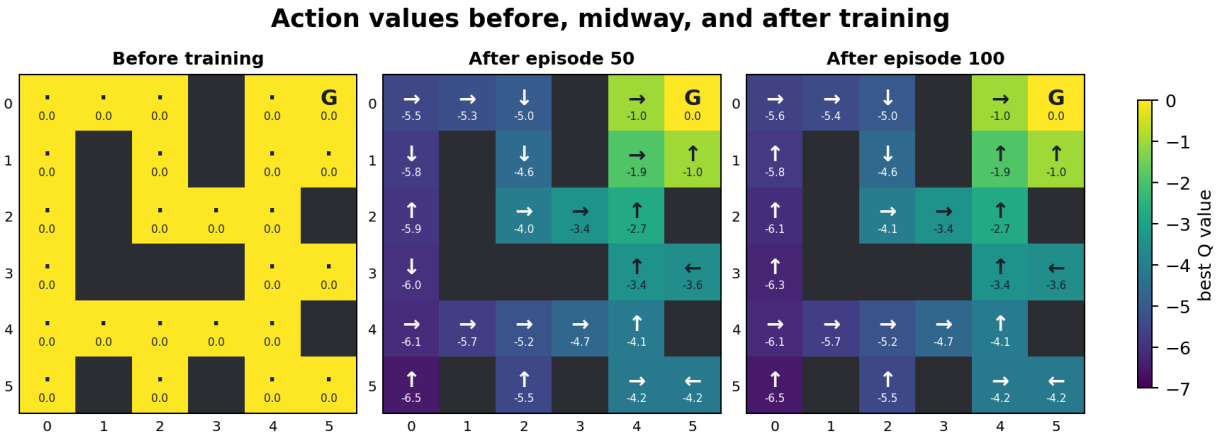


**Figure 3:** Best legal action and its Q-value before training, after episode 50, and after episode 100.

Before training, every entry is zero. Values near the goal are learned first, then the estimates move back through the maze. After episode 100, the best value at the start is $-5.6128$. The Q-value is not simply $-10$ because future rewards are discounted by $\gamma = 0.90$.

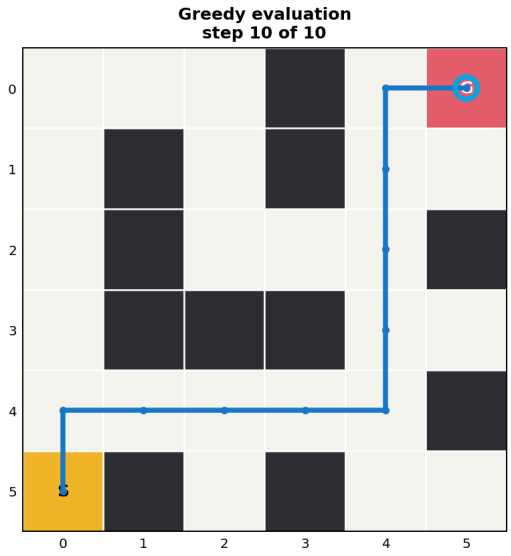


**Figure 4:** Greedy evaluation after training.

**Final check**
The greedy policy reaches the goal in 10 steps. It is therefore a shortest path for this layout. No exploration is used and the Q-table is left unchanged.

## 6 Conclusion

This report implements Q-learning loop in one small maze in Rust language. Epsilon-greedy exploration is used during training. The separate check with epsilon set to zero then tests the learned policy without random actions or further Q updates.

## References and reproduction

**Reference.** Y. Kobayashi, *Reinforcement Learning*, lecture slides, 11 August 2026,